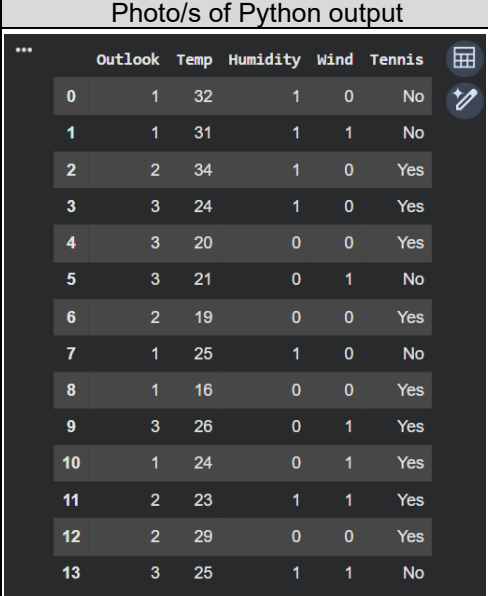
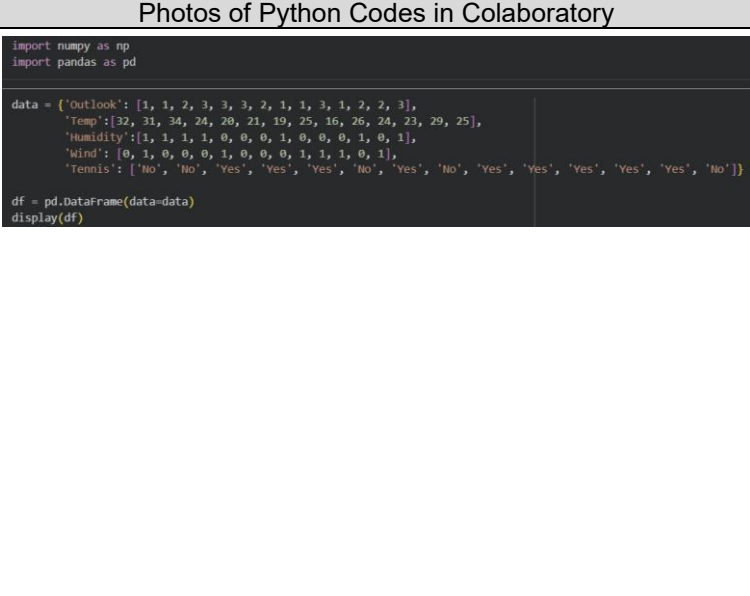


Name:	Lyka Sophia S. Jardeleza	Date:	01/29/2026
Program:	BSCpE	Section:	A461

A. Guided Exercises:

(Please include the following: 1. Photos of the codes and 2. Outputs after executing the Python code.)

1. Creating Data Frame

Photo/s of Python output	Photos of Python Codes in Colaboratory
	 <pre>import numpy as np import pandas as pd data = {'Outlook': [1, 1, 2, 3, 3, 3, 2, 1, 1, 3, 1, 2, 2, 3], 'Temp': [32, 31, 34, 24, 20, 21, 19, 25, 16, 26, 24, 23, 29, 25], 'Humidity': [1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1], 'Wind': [0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1], 'Tennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']} df = pd.DataFrame(data=data) display(df)</pre>

2. Manipulating Values in Data Frame AND Renaming, Adding, and Dropping a Data Column

Photo/s of Python output	Photos of Python Codes in Colaboratory																																																																																																								
Manipulating Values in Data Frame <pre>Outlook int64 Temp int64 Humidity int64 Wind int64 Tennis object dtype: object</pre><table><thead><tr><th></th><th>Outlook</th><th>Temp</th><th>Humidity</th><th>Wind</th><th>Tennis</th></tr></thead><tbody><tr><td>0</td><td>Sunny</td><td>32</td><td></td><td>1</td><td>0</td><td>No</td></tr><tr><td>1</td><td>Sunny</td><td>31</td><td></td><td>1</td><td>1</td><td>No</td></tr><tr><td>2</td><td>Overcast</td><td>34</td><td></td><td>1</td><td>0</td><td>Yes</td></tr><tr><td>3</td><td>Rain</td><td>24</td><td></td><td>1</td><td>0</td><td>Yes</td></tr><tr><td>4</td><td>Rain</td><td>20</td><td></td><td>0</td><td>0</td><td>Yes</td></tr><tr><td>5</td><td>Rain</td><td>21</td><td></td><td>0</td><td>1</td><td>No</td></tr><tr><td>6</td><td>Overcast</td><td>19</td><td></td><td>0</td><td>0</td><td>Yes</td></tr><tr><td>7</td><td>Sunny</td><td>25</td><td></td><td>1</td><td>0</td><td>No</td></tr><tr><td>8</td><td>Sunny</td><td>16</td><td></td><td>0</td><td>0</td><td>Yes</td></tr><tr><td>9</td><td>Rain</td><td>26</td><td></td><td>0</td><td>1</td><td>Yes</td></tr><tr><td>10</td><td>Sunny</td><td>24</td><td></td><td>0</td><td>1</td><td>Yes</td></tr><tr><td>11</td><td>Overcast</td><td>23</td><td></td><td>1</td><td>1</td><td>Yes</td></tr><tr><td>12</td><td>Overcast</td><td>29</td><td></td><td>0</td><td>0</td><td>Yes</td></tr><tr><td>13</td><td>Rain</td><td>25</td><td></td><td>1</td><td>1</td><td>No</td></tr></tbody></table>		Outlook	Temp	Humidity	Wind	Tennis	0	Sunny	32		1	0	No	1	Sunny	31		1	1	No	2	Overcast	34		1	0	Yes	3	Rain	24		1	0	Yes	4	Rain	20		0	0	Yes	5	Rain	21		0	1	No	6	Overcast	19		0	0	Yes	7	Sunny	25		1	0	No	8	Sunny	16		0	0	Yes	9	Rain	26		0	1	Yes	10	Sunny	24		0	1	Yes	11	Overcast	23		1	1	Yes	12	Overcast	29		0	0	Yes	13	Rain	25		1	1	No	<pre># Create a deep copy for the DataFrame df_copy = df.copy(deep=True) # show the datatypes of the DataFrame print(df_copy.dtypes, end='\n\n') # convert data type of Outlook column # from int to string df_copy.Outlook = df_copy.Outlook.astype(str) # Changing or manipulating the values in the DataFrame for i in range(len(df_copy['Outlook'])): val = df_copy['Outlook'].values[i] if val == "1": df_copy['Outlook'].values[i] = 'Sunny' elif val == "2": df_copy['Outlook'].values[i] = 'Overcast' else: df_copy['Outlook'].values[i] = 'Rain' display(df_copy)</pre>
	Outlook	Temp	Humidity	Wind	Tennis																																																																																																				
0	Sunny	32		1	0	No																																																																																																			
1	Sunny	31		1	1	No																																																																																																			
2	Overcast	34		1	0	Yes																																																																																																			
3	Rain	24		1	0	Yes																																																																																																			
4	Rain	20		0	0	Yes																																																																																																			
5	Rain	21		0	1	No																																																																																																			
6	Overcast	19		0	0	Yes																																																																																																			
7	Sunny	25		1	0	No																																																																																																			
8	Sunny	16		0	0	Yes																																																																																																			
9	Rain	26		0	1	Yes																																																																																																			
10	Sunny	24		0	1	Yes																																																																																																			
11	Overcast	23		1	1	Yes																																																																																																			
12	Overcast	29		0	0	Yes																																																																																																			
13	Rain	25		1	1	No																																																																																																			



Renaming, Adding, and Dropping a Data Column

	Outlook	DayofWeek	Temp	Humidity	Tennis
0	Sunny	1	32	1	No
1	Sunny	2	31	1	No
2	Overcast	3	34	1	Yes
3	Rain	4	24	1	Yes
4	Rain	5	20	0	Yes
5	Rain	6	21	0	No
6	Overcast	7	19	0	Yes
7	Sunny	1	25	1	No
8	Sunny	2	16	0	Yes
9	Rain	3	26	0	Yes
10	Sunny	4	24	0	Yes
11	Overcast	5	23	1	Yes
12	Overcast	6	29	0	Yes
13	Rain	7	25	1	No

```
# Rename the column Tennis to Basketball
df_new = df_copy.copy(deep=True)
df_neww = df_new.rename(columns={df.columns[4]:'Basketball'})

# Adding a new data column at index 1
df_new.insert(1, 'DayofWeek', [1, 2, 3, 4, 5, 6, 7, 1, 2, 3, 4, 5, 6, 7], False)

# Drop a data column
df_new.drop('Wind', axis=1, inplace=True)

display(df_new)
```

3. Creating Subsets From the Data Frame AND Statistical Operations with Column in Data Frame

Photo/s of Python output

Creating Subsets From the Data Frame

	Outlook	Temp	Humidity	Wind
0	1	32	1	0
1	1	31	1	1
2	2	34	1	0
3	3	24	1	0
4	3	20	0	0
5	3	21	0	1

Tennis

0	No
1	No
2	Yes
3	Yes
4	Yes
5	No
6	Yes
7	No

Statistical Operations with Column in Data Frame

```
Mean of Temp col: 24.928571428571427
Standard Deviation of Temp col: 5.165928105075911
Variance of Temp col: 26.686813186813193
Median of Outlook col: 2.0
Mode of Outlook col: 0    1
1    3
Name: Outlook, dtype: int64
```

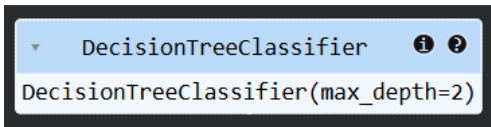
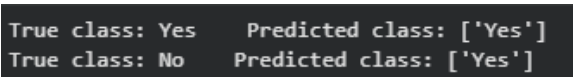
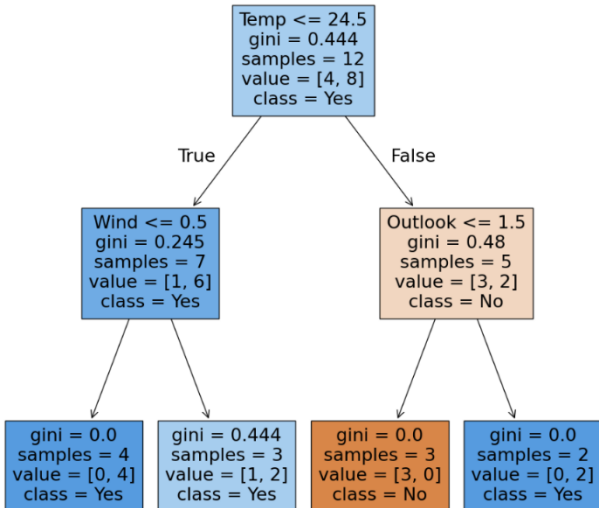
Photos of Python Codes in Colaboratory

```
# Subset of the original dataframe
# columns: 'Outlook', 'Temp', 'Humidity', 'Wind'
# rows: 0 to 5
sub_set = df.loc[0:5, ['Outlook', 'Temp', 'Humidity', 'Wind']]
display(sub_set)

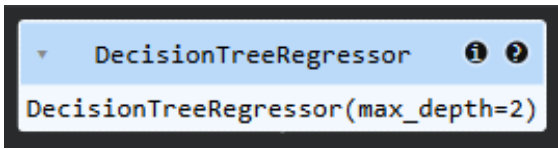
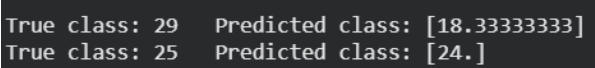
# Subset of the original dataframe
# columns: 'Tennis'
# rows: 0 to 7
classes = df.loc[0:7, ['Tennis']]
display(classes)
```

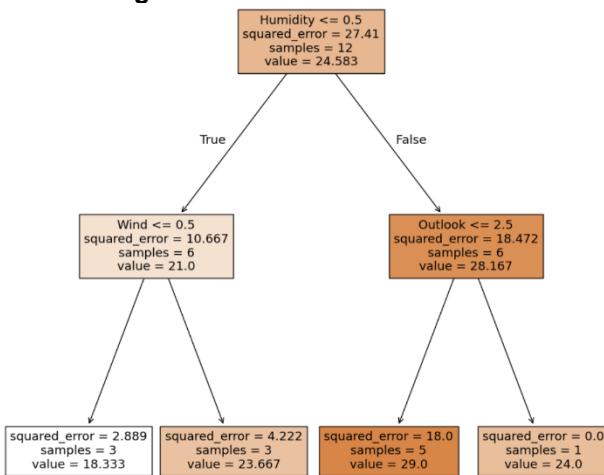
```
# Some statistical operations
# the with data frame columns
print("Mean of Temp col:", df.Temp.mean())
print("Standard Deviation of Temp col:", df.Temp.std())
print("Variance of Temp col:", df.Temp.var())
print("Median of Outlook col:", df.Outlook.median())
print("Mode of Outlook col:", df.Outlook.mode())
```

4. Growing, Prediction, and Visualizing Decision Tree

Photo/s of Python output	Photos of Python Codes in Colaboratory
Growing 	<pre>#Growing Decision Tress with Scikit-Learn from sklearn import tree decision_tree = tree.DecisionTreeClassifier(max_depth=2, min_samples_split=2) #Create training subsets of the data train_features = df.loc[0:11, ['Outlook', 'Temp', 'Humidity', 'Wind']] train_classes = df.loc[0:11, ['Tennis']] #Grow a decision tree based on the training data decision_tree.fit(train_features, train_classes)</pre>
Predicting 	<pre>#Predicting New Data with Decision Tree #Create test subsets of the data test_features = df.loc[12:13, ['Outlook', 'Temp', 'Humidity', 'Wind']] test_classes = df.loc[12:13, ['Tennis']] #Predict the test data print("True class:", test_classes['Tennis'].values[0], " Predicted class:", decision_tree.predict([test_features.iloc[0]])) print("True class:", test_classes['Tennis'].values[1], " Predicted class:", decision_tree.predict([test_features.iloc[1]]))</pre>
Visualizing 	<pre>#Visualizing the Decision Tree from matplotlib import pyplot as plt fig = plt.figure(figsize=(12, 12)) _ = tree.plot_tree(decision_tree, feature_names=list(train_features.keys()), class_names=['No', 'Yes'], filled=True)</pre>

5. Growing, Prediction, and Visualizing Regression Tree

Photo/s of Python output	Photos of Python Codes in Colaboratory
Growing 	<pre>#Growing Regression Trees with Scikit-Learn from sklearn import tree regression_tree = tree.DecisionTreeRegressor(max_depth=2, min_samples_split=2) #Create training subsets of the data reg_train_features = df.loc[0:11, ['Outlook', 'Humidity', 'Wind']] reg_train_targets = df.loc[0:11, ['Temp']] regression_tree.fit(reg_train_features, reg_train_targets)</pre>
Predicting 	<pre>#Predicting New Data with Regression tree #Create test subsets of the data reg_test_features = df.loc[12:13, ['Outlook', 'Humidity', 'Wind']] reg_test_targets = df.loc[12:13, ['Temp']] #Predict the test data print("True class:", reg_test_targets['Temp'].values[0], " Predicted class:", regression_tree.predict([reg_test_features.iloc[0]])) print("True class:", reg_test_targets['Temp'].values[1], " Predicted class:", regression_tree.predict([reg_test_features.iloc[1]]))</pre>

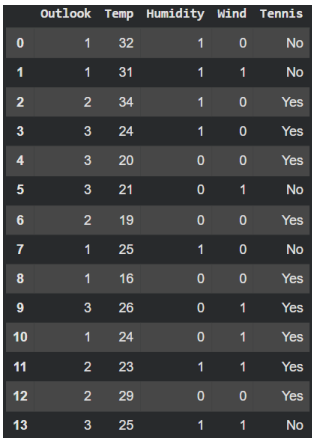
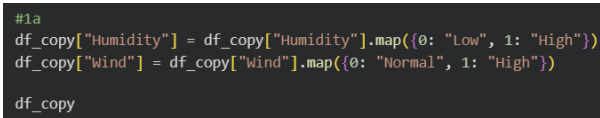
Visualizing


```
#Visualizing the Regression Tree
from matplotlib import pyplot as plt
fig = plt.figure(figsize=(12, 12))
_ = tree.plot_tree(regression_tree, feature_names=list(reg_train_features.keys()), filled=True)
```

B. Problem Exercises:

(Please include the following: 1. Photos of the codes and 2. Outputs after executing the Python code.)

Problem 1:

Photo/s of Python output	Photos of Python Codes in Colaboratory
 <pre> Outlook Temp Humidity Wind Tennis 0 1 32 1 0 No 1 1 31 1 1 No 2 2 34 1 0 Yes 3 3 24 1 0 Yes 4 3 20 0 0 Yes 5 3 21 0 1 No 6 2 19 0 0 Yes 7 1 25 1 0 No 8 1 16 0 0 Yes 9 3 26 0 1 Yes 10 1 24 0 1 Yes 11 2 23 1 1 Yes 12 2 29 0 0 Yes 13 3 25 1 1 No </pre>	 <pre> #Coding Problem 1 import pandas as pd import numpy as np from math import log2 from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree import matplotlib.pyplot as plt #Create the dataset data = { "Outlook": [1,1,2,3,3,3,2,1,1,3,1,2,2,3], "Temp": [32,31,34,24,20,21,19,25,16,26,24,23,29,25], "Humidity": [1,1,1,0,0,0,1,0,0,0,1,0,1], "Wind": [0,1,0,0,0,1,0,0,0,1,1,1,0,1], "Tennis": ["No","No","Yes","Yes","Yes","No","Yes","No", "Yes","Yes","Yes","Yes","Yes","No"] } df = pd.DataFrame(data) df_copy = df.copy() df </pre>
a)  <pre> Outlook Temp Humidity Wind Tennis 0 1 32 High Normal No 1 1 31 High High No 2 2 34 High Normal Yes 3 3 24 High Normal Yes 4 3 20 Low Normal Yes 5 3 21 Low High No 6 2 19 Low Normal Yes 7 1 25 High Normal No 8 1 16 Low Normal Yes 9 3 26 Low High Yes 10 1 24 Low High Yes 11 2 23 High High Yes 12 2 29 Low Normal Yes 13 3 25 High High No </pre>	a)  <pre> #1a df_copy["Humidity"] = df_copy["Humidity"].map({0: "Low", 1: "High"}) df_copy["Wind"] = df_copy["Wind"].map({0: "Normal", 1: "High"}) df_copy </pre>

b)

	Outlook	Temp	Humidity	Wind	Tennis
0	1	32	1	0	No
1	1	31	1	1	No
2	2	34	1	0	Yes
3	3	24	1	0	Yes
4	3	20	0	0	Yes
5	3	21	0	1	No
6	2	19	0	0	Yes
7	1	25	1	0	No
8	1	16	0	0	Yes
9	3	26	0	1	Yes
10	1	24	0	1	Yes
11	2	23	1	1	Yes
12	2	29	0	0	Yes
13	3	25	1	1	No
14	2	28	1	1	No
15	3	22	0	1	Yes
16	1	19	0	0	Yes

c)

```
np.float64(0.45674740484429055)
```

d)

```
np.float64(0.9366673818775626)
```

e)

```
('Temp', np.int64(24), np.float64(0.3251633986928104))
```

b)

```
#1b
new_rows = pd.DataFrame({
    "Outlook": [2,3,1],
    "Temp": [28,22,19],
    "Humidity": [1,0,0],
    "Wind": [1,1,0],
    "Tennis": ["No","Yes","Yes"]
})

df = pd.concat([df, new_rows], ignore_index=True)
df
```

c)

```
#1c
def gini_impurity(target):
    values, counts = np.unique(target, return_counts=True)
    prob = counts / counts.sum()
    return 1 - np.sum(prob ** 2)

gini_impurity(df["Tennis"])
```

d)

```
#1d
def entropy(target):
    values, counts = np.unique(target, return_counts=True)
    prob = counts / counts.sum()
    return -np.sum(prob * np.log2(prob))

entropy(df["Tennis"])
```

e)

```
#1e
def best_split_gini(df, target_col):
    best_feature = None
    best_value = None
    best_gini = float("inf")

    for feature in df.columns:
        if feature == target_col:
            continue

        values = df[feature].unique()
        for v in values:
            left = df[df[feature] <= v]
            right = df[df[feature] > v]

            if len(left) == 0 or len(right) == 0:
                continue

            gini = (
                len(left)/len(df) * gini_impurity(left[target_col]) +
                len(right)/len(df) * gini_impurity(right[target_col])
            )

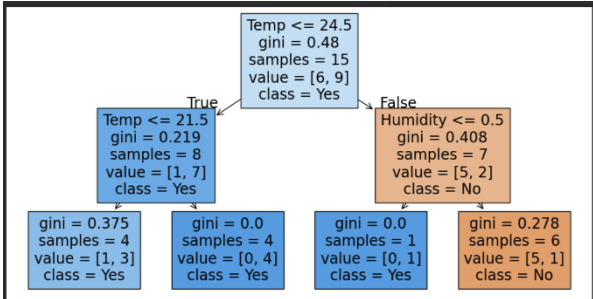
            if gini < best_gini:
                best_gini = gini
                best_feature = feature
                best_value = v

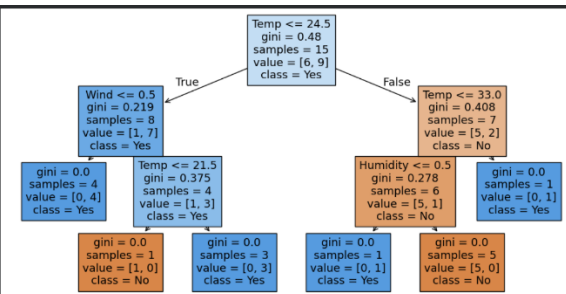
    return best_feature, best_value, best_gini

best_split_gini(df, "Tennis")
```

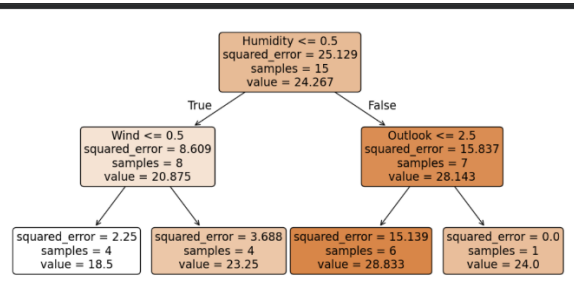
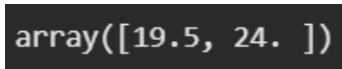
f) <pre>np.float64(0.22109108561988422)</pre>	f) <pre>#1f def information_gain(df, feature, value, target_col): total_entropy = entropy(df[target_col]) left = df[df[feature] <= value] right = df[df[feature] > value] weighted_entropy = (len(left)/len(df) * entropy(left[target_col]) + len(right)/len(df) * entropy(right[target_col])) return total_entropy - weighted_entropy feature, value, _ = best_split_gini(df, "Tennis") information_gain(df, feature, value, "Tennis")</pre>
--	--

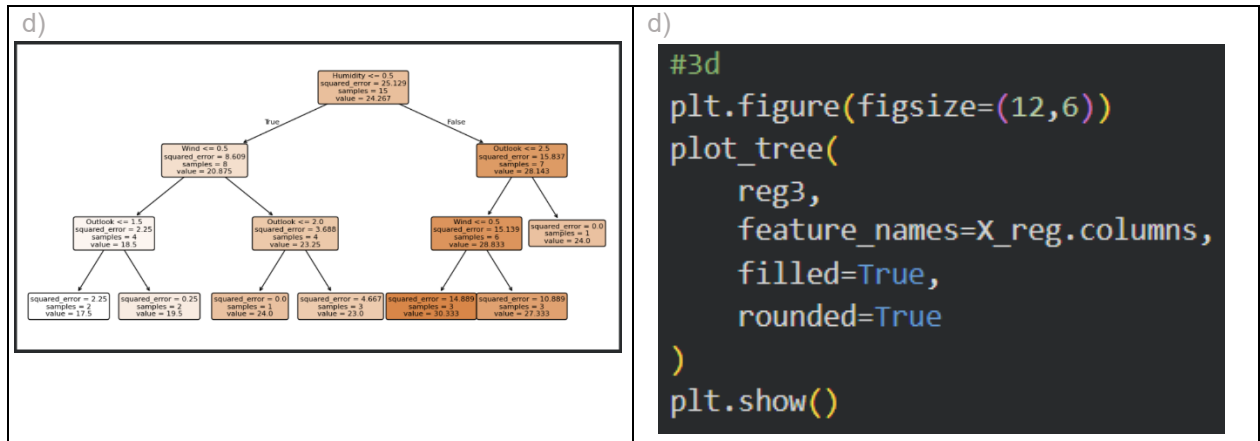
Problem 2:

Photo/s of Python output	Photos of Python Codes in Colaboratory
a) <pre>array(['Yes', 'No'], dtype=object)</pre> b)  <pre> graph TD Node1["Temp <= 24.5 gini = 0.48 samples = 15 value = [6, 9] class = Yes"] Node2["Temp <= 21.5 gini = 0.219 samples = 8 value = [1, 7] class = Yes"] Node3["Humidity <= 0.5 gini = 0.408 samples = 7 value = [5, 2] class = No"] Node4["gini = 0.375 samples = 4 value = [1, 3] class = Yes"] Node5["gini = 0.0 samples = 4 value = [0, 4] class = Yes"] Node6["gini = 0.0 samples = 1 value = [0, 1] class = Yes"] Node7["gini = 0.278 samples = 6 value = [5, 1] class = No"] Node1 -- True --> Node2 Node1 -- False --> Node3 Node2 --> Node4 Node2 --> Node5 Node3 --> Node6 Node3 --> Node7 </pre> c) <pre>array(['Yes', 'Yes'], dtype=object)</pre>	a) <pre># Coding Problem 2 x = df.drop("Tennis", axis=1) y = df["Tennis"] train_idx = list(range(0,8)) + list(range(10,17)) test_idx = [8,9] X_train, X_test = x.iloc[train_idx], x.iloc[test_idx] y_train, y_test = y.iloc[train_idx], y.iloc[test_idx] #2a clf2 = DecisionTreeClassifier(max_depth=2) clf2.fit(X_train, y_train) pred2 = clf2.predict(X_test) pred2</pre> b) <pre>#2b plt.figure(figsize=(12,6)) plot_tree(clf2, feature_names=X.columns, class_names=["No", "Yes"], filled=True) plt.show()</pre> c) <pre>#2c clf3 = DecisionTreeClassifier(max_depth=3) clf3.fit(X_train, y_train) pred3 = clf3.predict(X_test) pred3</pre>

d) 	d) <pre>#2d plt.figure(figsize=(12,6)) plot_tree(clf3, feature_names=X.columns, class_names=["No","Yes"], filled=True) plt.show()</pre>
---	--

Problem 3:

Photo/s of Python output	Photos of Python Codes in Colaboratory
a)  b)  c) 	a) <pre># Coding Problem 3 from sklearn.tree import DecisionTreeRegressor, plot_tree import matplotlib.pyplot as plt X_reg = df.drop(columns=["Temp", "Tennis"]) y_reg = df["Temp"] train_idx = list(range(0, 12)) + list(range(14, 17)) test_idx = [12, 13] X_train_r = X_reg.iloc[train_idx] X_test_r = X_reg.iloc[test_idx] y_train_r = y_reg.iloc[train_idx] y_test_r = y_reg.iloc[test_idx] #3a reg2 = DecisionTreeRegressor(max_depth=2, random_state=0) reg2.fit(X_train_r, y_train_r) pred_r2 = reg2.predict(X_test_r) pred_r2</pre> b) <pre>#3b plt.figure(figsize=(12,6)) plot_tree(reg2, feature_names=X_reg.columns, filled=True, rounded=True) plt.show()</pre> c) <pre>#3c reg3 = DecisionTreeRegressor(max_depth=3, random_state=0) reg3.fit(X_train_r, y_train_r) pred_r3 = reg3.predict(X_test_r) pred_r3</pre>



C. Findings ,Observations, and Comments:

Guiding Questions :

1. What did you learn in the guided exercises? Did you learn something new?
2. What are your findings, observations, and comments in the guided exercises?
3. What are your findings, observations, and comments in the problem exercises?
4. Did you experience any difficulties in solving the problem exercises?

I learned how to work with datasets and apply basic decision trees and yes, I learned new concepts and steps I was not familiar with before.

The guided exercises helped me understand the flow of the tasks better. The instructions were clear, and the results matched the expected output.

In the problem exercises, I observed that applying what was learned required more attention and practice, but it helped reinforce the lessons we had in our lecture.

Yes, I experienced some difficulties at first, especially in applying the steps correctly, but reviewing the examples helped resolve them.